CSSE 230

Homework 7 Solutions

2. (15 points) Weiss Exercise 20.5.

	Linear		Quadratic		Separate Chaining
0	9679	0	9679	0	
1	4371	1	4371	1	4371
2	1989	2		2	
3	1323	3	1323	3	1323, 6173
4	6173	4	6173	4	4344
5	4344	5	4344	5	
6		6		6	
7		7		7	
8		8	1989	8	
9	4199	9	4199	9	4199, 9679, 1989

Grading: 5 points for each column. If something is wrong, and you can figure out what the student might have been thinking that led to a given error, assign partial credit based on how far off that thinking is. Otherwise give 0-2 points for an incorrect column.

```java
public class StringHashSet {

    private static final int DEFAULT_CAPACITY = 5;
    private int mCapacity;
    private int mSize;
    private Node[] mArray;

    private void initialize(int initialCapacity) {
        mCapacity = initialCapacity;
        mArray = new Node[mCapacity];
        mSize = 0;
    }

    public static int stringHashCode(String item) {
        int hash = 0;
        for (int i = 0; i < item.length(); i++) {
            hash = 31 * hash + item.charAt(i);
        }
        return hash;
    }

    private int getIndex(int hashCode, int capacity) {
        if (hashCode < 0) {
            hashCode += Integer.MAX_VALUE;
        }
        return hashCode % capacity;
    }

    public boolean add(String item) {
        if (contains(item)) {
            return false;
        }

        // Ignoring rehash
        int index = getIndex(stringHashCode(item), mCapacity);
        mArray[index] = new Node(item, mArray[index]);
        mSize++;
        return true;
    }
```

```java
public String toRawString() {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < mCapacity; i++) {
        sb.append(String.format("%d: %s\n", i, nodeString(mArray[i])));
    }
    return sb.toString();
}

public boolean contains(String item) {
    int index = getIndex(stringHashCode(item), mCapacity);
    Node current = mArray[index];
    while (current != null) {
        if (current.data.equals(item)) {
            return true;
        }
        current = current.next;
    }
    return false;
}

public int size() {
    return mSize;
}

public boolean isEmpty() {
    return size() == 0;
}

public void clear() {
    initialize(DEFAULT_CAPACITY);
}
```

```
public boolean remove(String item) {
    if (!contains(item)) {
        return false;
    }
    mSize--;
    int index = getIndex(stringHashCode(item), mCapacity);
    Node current = mArray[index];
    Node previous = null;
    while (current != null) {
        if (current.data.equals(item)) {
            // Remove this node. 2 cases to consider.
            if (previous == null) {
                mArray[index] = current.next;
            } else {
                previous.next = current.next;
            }
            return true;
        }
        previous = current;
        current = current.next;
    }
    return true;
}

public boolean addAll(Collection<String> collection) {
    boolean changed = false;
    for (String s : collection) {
        changed = add(s) || changed;
    }
    return changed;
}

private class Node {
    private String data;
    Node next;

    public Node(String item, Node next) {
        this.data = item;
        this.next = next;
    }

}

private String nodeString(Node n) {
    if (n == null) {
        return "null";
```